CS 229, Fall 2018

Problem Set #2: Supervised Learning II

Due Wednesday, Oct 31 at 11:59 pm on Gradescope.

Notes: (1) These questions require thought, but do not require long answers. Please be as concise as possible. (2) If you have a question about this homework, we encourage you to post your question on our Piazza forum, at `http://piazza.com/stanford/fall2018/cs229`. (3) If you missed the first lecture or are unfamiliar with the collaboration or honor code policy, please read the policy on Handout #1 (available from the course website) before starting work. (4) For the coding problems, you may not use any libraries except those defined in the provided `environment.yml` file. In particular, ML-specific libraries such as scikit-learn are not permitted. (5) To account for late days, the due date listed on Gradescope is Nov 03 at 11:59 pm. If you submit after Oct 31, you will begin consuming your late days. If you wish to submit on time, submit before Oct 31 at 11:59 pm.

All students must submit an electronic PDF version of the written questions. We highly recommend typesetting your solutions via LaTeX. If you are scanning your document by cell phone, please check the Piazza forum for recommended scanning apps and best practices. All students must also submit a zip file of their source code to Gradescope, which should be created using the `make_zip.py` script. In order to pass the auto-grader tests, you should make sure to (1) restrict yourself to only using libraries included in the `environment.yml` file, and (2) make sure your code runs without errors when running `p05_percept.py` and `p06_spam.py`. Your submission will be evaluated by the auto-grader using a private test set.

1. [15 points] Logistic Regression: Training stability

In this problem, we will be delving deeper into the workings of logistic regression. The goal of this problem is to help you develop your skills debugging machine learning algorithms (which can be very different from debugging software in general).

We have provided a implementation of logistic regression in `src/p01_lr.py`, and two labeled datasets A and B in `data/ds1_a.txt` and `data/ds1_b.txt`.

Please do not modify the code for the logistic regression training algorithm for this problem. First, run the given logistic regression code to train two different models on A and B. You can run the code by simply executing `pythonp01_lr.py` in the `src` directory.

(a) **[2 points]** What is the most notable difference in training the logistic regression model on datasets A and B?

(b) **[5 points]** Investigate why the training procedure behaves unexpectedly on dataset B, but not on A. Provide hard evidence (in the form of math, code, plots, etc.) to corroborate your hypothesis for the misbehavior. Remember, you should address why your explanation does not apply to A.

Hint: The issue is not a numerical rounding or over/underflow error.

(c) **[5 points]** For each of these possible modifications, state whether or not it would lead to the provided training algorithm converging on datasets such as B. Justify your answers.

i. Using a different constant learning rate.

ii. Decreasing the learning rate over time (e.g. scaling the initial learning rate by $1/t^2$, where t is the number of gradient descent iterations thus far).

iii. Linear scaling of the input features.

iv. Adding a regularization term $\|\theta\|_2^2$ to the loss function.

v. Adding zero-mean Gaussian noise to the training data or labels.

(d) **[3 points]** Are support vector machines, which use the hinge loss, vulnerable to datasets like B? Why or why not? Give an informal justification.

Hint: Recall the distinction between functional margin and geometric margin.

2. [10 points] Model Calibration

In this question we will try to understand the output $h_\theta(x)$ of the hypothesis function of a logistic regression model, in particular why we might treat the output as a probability (besides the fact that the sigmoid function ensures $h_\theta(x)$ always lies in the interval $(0, 1)$).

When the probabilities outputted by a model match empirical observation, the model is said to be well-calibrated (or reliable). For example, if we consider a set of examples $x^{(i)}$ for which $h_\theta(x^{(i)}) \approx 0.7$, around 70% of those examples should have positive labels. In a well-calibrated model, this property will hold true at every probability value.

Logistic regression tends to output well-calibrated probabilities (this is often not true with other classifiers such as Naive Bayes, or SVMs). We will dig a little deeper in order to understand why this is the case, and find that the structure of the loss function explains this property.

Suppose we have a training set $\{x^{(i)}, y^{(i)}\}_{i=1}^m$ with $x^{(i)} \in \mathbb{R}^{n+1}$ and $y^{(i)} \in \{0, 1\}$. Assume we have an intercept term $x_0^{(i)} = 1$ for all i. Let $\theta \in \mathbb{R}^{n+1}$ be the maximum likelihood parameters learned after training a logistic regression model. In order for the model to be considered well-calibrated, given any range of probabilities (a, b) such that $0 \leq a < b \leq 1$, and training examples $x^{(i)}$ where the model outputs $h_\theta(x^{(i)})$ fall in the range (a, b), the fraction of positives in that set of examples should be equal to the average of the model outputs for those examples. That is, the following property must hold:

$$\frac{\sum_{i \in I_{a,b}} P\big(y^{(i)} = 1 \mid x^{(i)}; \theta\big)}{|\{i \in I_{a,b}\}|} = \frac{\sum_{i \in I_{a,b}} \mathbb{I}\{y^{(i)} = 1\}}{|\{i \in I_{a,b}\}|},$$

where $P(y = 1 \mid x; \theta) = h_\theta(x) = 1/(1 + \exp(-\theta^\top x))$, $I_{a,b} = \{i \mid i \in \{1, \ldots, m\}, h_\theta(x^{(i)}) \in (a, b)\}$ is an index set of all training examples $x^{(i)}$ where $h_\theta(x^{(i)}) \in (a, b)$, and $|S|$ denotes the size of the set S.

(a) **[5 points]** Show that the above property holds true for the described logistic regression model over the range $(a, b) = (0, 1)$.

Hint: Use the fact that we include a bias term.

(b) **[3 points]** If we have a binary classification model that is perfectly calibrated—that is, the property we just proved holds for any $(a, b) \subset [0, 1]$—does this necessarily imply that the model achieves perfect accuracy? Is the converse necessarily true? Justify your answers.

(c) **[2 points]** Discuss what effect including L_2 regularization in the logistic regression objective has on model calibration.

Remark: We considered the range $(a, b) = (0, 1)$. This is the only range for which logistic regression is guaranteed to be calibrated on the training set. When the GLM modeling assumptions hold, all ranges $(a, b) \subset [0, 1]$ are well calibrated. In addition, when the training and test set are from the same distribution and when the model has not overfit or underfit, logistic regression tends to be well-calibrated on unseen test data as well. This makes logistic regression a very popular model in practice, especially when we are interested in the level of uncertainty in the model output.

3. [20 points] Bayesian Interpretation of Regularization

Background: In Bayesian statistics, almost every quantity is a random variable, which can either be observed, or unobserved. For instance, parameters θ are generally unobserved random variables, and data x and y are observed random variables. The joint distribution of all the random variables is also called the model (e.g $p(x, y, \theta)$). Every unknown quantity can be estimated by conditioning the model on all the observed quantities. Such a conditional distribution over the unobserved random variables, conditioned on the observed random variables, is called the posterior distribution. For instance $p(\theta \mid x, y)$ is the posterior distribution in the machine learning context. A consequence of this approach is that, we are required to endow our model parameters, i.e. $p(\theta)$, with a prior distribution. The prior probabilities are to be assigned before we see the data – they need to capture our prior beliefs of what the model parameters might be before observing any evidence, and must be a subjective opinion by the person building the model.

In the purest Bayesian interpretation, we are required to keep the entire posterior distribution over the parameters all the way until prediction, to come up with the posterior predictive distribution, and the final prediction will be the expected value of the posterior predictive distribution. However in most situations, this is computationally very expensive, and we settle for a compromise that is less pure (in the Bayesian sense).

The compromise is to estimate a point value of the parameters (instead of the full distribution) which is the mode of the posterior distribution. Estimating the mode of the posterior distribution is also called maximum a posteriori estimation (MAP). That is,

$$\theta_{\text{MAP}} = \arg\max_{\theta} p(\theta \mid x, y).$$

Compare this to the maximum likelihood estimation (MLE) we have seen previously:

$$\theta_{\text{MLE}} = \arg\max_{\theta} p(y \mid x, \theta).$$

In this problem, we explore the connection between MAP estimation, and common regularization techniques that are applied with MLE estimation. In particular, you will show how the choice of prior distribution over θ (e.g Gaussian, or Laplace prior) is equivalent to different kinds of regularization (e.g L_2, or L_1 regularization). To show this, we shall proceed step by step, showing intermediate steps.

(a) **[3 points]** Show that $\theta_{\text{MAP}} = \arg\max_\theta p(y \mid x, \theta)p(\theta)$ if we assume that $p(\theta) = p(\theta \mid x)$. The assumption that $p(\theta) = p(\theta \mid x)$ will be valid for models such as linear regression where the input x are not explicitly modeled by θ. (Note that this means x and θ are marginally independent, but not conditionally independent when y is given.)

(b) **[5 points]** Recall that L_2 regularization penalizes the L_2 norm of the parameters while minimizing the loss (i.e., negative log likelihood in case of probabilistic models). Now we will show that MAP estimation with a zero-mean Gaussian prior over θ, specifically $\theta \sim \mathcal{N}(0, \eta^2 I)$, is equivalent to applying L_2 regularization with MLE estimation. Specifically, show that

$$\theta_{\text{MAP}} = \arg\min_{\theta} \left[-\log p(y \mid x, \theta) + \lambda \|\theta\|_2^2 \right].$$

Also, what is the value of λ?

(c) **[7 points]** Now consider a specific instance, a linear regression model given by $y = \theta^T x + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Like before, assume a Gaussian prior on this model such that $\theta \sim \mathcal{N}(0, \eta^2 I)$. For

notation, let X be the design matrix of all the training example inputs where each row vector is one example input, and $\vec{y}$ be the column vector of all the example outputs.

Come up with a closed form expression for θ_{MAP}.

(d) **[5 points]** Next, consider the Laplace distribution, whose density is given by

$$f_L(z \mid \mu, b) = \frac{1}{2b} \exp\left(-\frac{|z-\mu|}{b}\right).$$

As before, consider a linear regression model given by $y = x^T\theta + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Assume a Laplace prior on this model where $\theta \sim \mathcal{L}(0, bI)$.

Show that θ_{MAP} in this case is equivalent to the solution of linear regression with L_1 regularization, whose loss is specified as

$$J(\theta) = \|X\theta - \vec{y}\|_2^2 + \gamma\|\theta\|_1.$$

Also, what is the value of γ?

Note: A closed form solution for linear regression problem with L_1 regularization does not exist. To optimize this, we use gradient descent with a random initialization and solve it numerically.

Remark: Linear regression with L_2 regularization is also commonly called Ridge regression, and when L_1 regularization is employed, is commonly called Lasso regression. These regularizations can be applied to any Generalized Linear models just as above (by replacing $\log p(y \mid x, \theta)$ with the appropriate family likelihood). Regularization techniques of the above type are also called weight decay, and shrinkage. The Gaussian and Laplace priors encourage the parameter values to be closer to their mean (i.e., zero), which results in the shrinkage effect.

Remark: Lasso regression (i.e L_1 regularization) is known to result in sparse parameters, where most of the parameter values are zero, with only some of them non-zero.

4. [18 points] Constructing kernels

In class, we saw that by choosing a kernel $K(x, z) = \phi(x)^T\phi(z)$, we can implicitly map data to a high dimensional space, and have the SVM algorithm work in that space. One way to generate kernels is to explicitly define the mapping ϕ to a higher dimensional space, and then work out the corresponding K.

However in this question we are interested in direct construction of kernels. I.e., suppose we have a function $K(x, z)$ that we think gives an appropriate similarity measure for our learning problem, and we are considering plugging K into the SVM as the kernel function. However for $K(x, z)$ to be a valid kernel, it must correspond to an inner product in some higher dimensional space resulting from some feature mapping ϕ. Mercer's theorem tells us that $K(x, z)$ is a (Mercer) kernel if and only if for any finite set $\{x^{(1)}, \ldots, x^{(m)}\}$, the square matrix $K \in \mathbb{R}^{m\times m}$ whose entries are given by $K_{ij} = K(x^{(i)}, x^{(j)})$ is symmetric and positive semidefinite. You can find more details about Mercer's theorem in the notes, though the description above is sufficient for this problem.

Now here comes the question: Let K_1, K_2 be kernels over $\mathbb{R}^n \times \mathbb{R}^n$, let $a \in \mathbb{R}^+$ be a positive real number, let $f : \mathbb{R}^n \mapsto \mathbb{R}$ be a real-valued function, let $\phi : \mathbb{R}^n \to \mathbb{R}^d$ be a function mapping from $\mathbb{R}^n$ to $\mathbb{R}^d$, let K_3 be a kernel over $\mathbb{R}^d \times \mathbb{R}^d$, and let $p(x)$ a polynomial over x with positive coefficients.

For each of the functions K below, state whether it is necessarily a kernel. If you think it is, prove it; if you think it isn't, give a counter-example.

(a) **[1 points]** $K(x, z) = K_1(x, z) + K_2(x, z)$

(b) **[1 points]** $K(x, z) = K_1(x, z) - K_2(x, z)$

(c) **[1 points]** $K(x, z) = aK_1(x, z)$

(d) **[1 points]** $K(x, z) = -aK_1(x, z)$

(e) **[5 points]** $K(x, z) = K_1(x, z)K_2(x, z)$

(f) **[3 points]** $K(x, z) = f(x)f(z)$

(g) **[3 points]** $K(x, z) = K_3(\phi(x), \phi(z))$

(h) **[3 points]** $K(x, z) = p(K_1(x, z))$

[Hint: For part (e), the answer is that K is indeed a kernel. You still have to prove it, though. (This one may be harder than the rest.) This result may also be useful for another part of the problem.]

5. [16 points] Kernelizing the Perceptron

Let there be a binary classification problem with $y \in \{0, 1\}$. The perceptron uses hypotheses of the form $h_\theta(x) = g(\theta^T x)$, where $g(z) = \text{sign}(z) = 1$ if $z \geq 0$, 0 otherwise. In this problem we will consider a stochastic gradient descent-like implementation of the perceptron algorithm where each update to the parameters θ is made using only one training example. However, unlike stochastic gradient descent, the perceptron algorithm will only make one pass through the entire training set. The update rule for this version of the perceptron algorithm is given by

$$\theta^{(i+1)} := \theta^{(i)} + \alpha\big(y^{(i+1)} - h_{\theta^{(i)}}(x^{(i+1)})\big)x^{(i+1)}$$

where $\theta^{(i)}$ is the value of the parameters after the algorithm has seen the first i training examples. Prior to seeing any training examples, $\theta^{(0)}$ is initialized to $\vec{0}$.

(a) **[9 points]** Let K be a Mercer kernel corresponding to some very high-dimensional feature mapping ϕ. Suppose ϕ is so high-dimensional (say, ∞-dimensional) that it's infeasible to ever represent $\phi(x)$ explicitly. Describe how you would apply the "kernel trick" to the perceptron to make it work in the high-dimensional feature space ϕ, but without ever explicitly computing $\phi(x)$.

[Note: You don't have to worry about the intercept term. If you like, think of ϕ as having the property that $\phi_0(x) = 1$ so that this is taken care of.] Your description should specify:

i. **[3 points]** How you will (implicitly) represent the high-dimensional parameter vector $\theta^{(i)}$, including how the initial value $\theta^{(0)} = 0$ is represented (note that $\theta^{(i)}$ is now a vector whose dimension is the same as the feature vectors $\phi(x)$);

ii. **[3 points]** How you will efficiently make a prediction on a new input $x^{(i+1)}$. I.e., how you will compute $h_{\theta^{(i)}}(x^{(i+1)}) = g\big(\theta^{(i)T}\phi(x^{(i+1)})\big)$, using your representation of $\theta^{(i)}$; and

iii. **[3 points]** How you will modify the update rule given above to perform an update to θ on a new training example $(x^{(i+1)}, y^{(i+1)})$; i.e., using the update rule corresponding to the feature mapping ϕ:

$$\theta^{(i+1)} := \theta^{(i)} + \alpha\big(y^{(i+1)} - h_{\theta^{(i)}}(x^{(i+1)})\big)x^{(i+1)}$$

(b) **[5 points]** Implement your approach by completing the `initial_state`, `predict`, and `update_state` methods of `src/p05_percept.py`.

(c) **[2 points]** Run `src/p05_percept.py` to train kernelized perceptrons on `data/ds5_train.csv`. The code will then test the perceptron on `data/ds5_test.csv` and save the resulting predictions in the `src/output` folder. Plots will also be saved in `src/output`. We provide two kernels, a dot product kernel and an radial basis function (rbf) kernel. One of the provided kernels performs extremely poorly in classifying the points. Which kernel performs badly and why does it fail?

6. [22 points] Spam classification

In this problem, we will use the naive Bayes algorithm and an SVM to build a spam classifier. In recent years, spam on electronic media has been a growing concern. Here, we'll build a classifier to distinguish between real messages, and spam messages. For this class, we will be building a classifier to detect SMS spam messages. We will be using an SMS spam dataset developed by Tiago A. Almedia and José María Gómez Hidalgo which is publicly available on `http://www.dt.fee.unicamp.br/~tiago/smsspamcollection`[1]

We have split this dataset into training and testing sets and have included them in this assignment as `data/ds6_spam_train.tsv` and `data/ds6_spam_test.tsv`. See `data/ds6_readme.txt` for more details about this dataset. Please refrain from redistributing these dataset files. The goal of this assignment is to build a classifier from scratch that can tell the difference the spam and non-spam messages using the text of the SMS message.

(a) **[5 points]** Implement code for processing the the spam messages into numpy arrays that can be fed into machine learning models. Do this by completing the `get_words`, `create_dictionary`, and `transform_text` functions within our provided `src/p06_spam.py`. Do note the corresponding comments for each function for instructions on what specific processing is required.

The provided code will then run your functions and save the resulting dictionary into `output/p06_dictionary` and a sample of the resulting training matrix into `output/p06_sample_train_matrix`.

(b) **[10 points]** In this question you are going to implement a naive Bayes classifier for spam classification with multinomial event model and Laplace smoothing (refer to class notes on Naive Bayes for details on Laplace smoothing).

Write your implementation by completing the `fit_naive_bayes_model` and `predict_from_naive_bayes_model` functions in `src/p06_spam.py`.

`src/p06_spam.py` should then be able to train a Naive Bayes model, compute your prediction accuracy and then save your resulting predictions to `output/p06_naive_bayes_predictions`.

Remark. If you implement naive Bayes the straightforward way, you'll find that the computed $p(x \mid y) = \prod_i p(x_i \mid y)$ often equals zero. This is because $p(x \mid y)$, which is the product of many numbers less than one, is a very small number. The standard computer representation of real numbers cannot handle numbers that are too small, and instead rounds them off to zero. (This is called "underflow.") You'll have to find a way to compute Naive Bayes' predicted class labels without explicitly representing very small numbers such as $p(x \mid y)$. [Hint: Think about using logarithms.]

(c) **[5 points]** Intuitively, some tokens may be particularly indicative of an SMS being in a particular class. We can try to get an informal sense of how indicative token i is for the SPAM class by looking at:

$$\log \frac{p(x_j = i \mid y = 1)}{p(x_j = i \mid y = 0)} = \log \left(\frac{P(\text{token } i \mid \text{email is SPAM})}{P(\text{token } i \mid \text{email is NOTSPAM})} \right).$$

Complete the `get_top_five_naive_bayes_words` function within the provided code using the above formula in order to obtain the 5 most indicative tokens.

The provided code will print out the resulting indicative tokens and then save them to `output/p06_top_indicative_words`.

[1] Almeida, T.A., Gómez Hidalgo, J.M., Yamakami, A. Contributions to the Study of SMS Spam Filtering: New Collection and Results. Proceedings of the 2011 ACM Symposium on Document Engineering (DOCENG'11), Mountain View, CA, USA, 2011.

(d) **[2 points]** Support vector machines (SVMs) are an alternative machine learning model that we discussed in class. We have provided you an SVM implementation (using a radial basis function (RBF) kernel) within `src/svm.py` (You should not need to modify that code). One important part of training an SVM parameterized by an RBF kernel is choosing an appropriate kernel radius.

Complete the `compute_best_svm_radius` by writing code to compute the best SVM radius which maximizes accuracy on the validation dataset.

The provided code will use your `compute_best_svm_radius` to compute and then write the best radius into `output/p06_optimal_radius`.